

Ejercicio 3

Descargar este enunciado [pdf](#) o [html](#)

Partiendo del [Ejercicio 2](#), se pide realizar las siguientes tareas:

Completa el código de `FichaCocheScreen.kt`, que muestra la ficha de un coche, para que en la parte derecha del precio muestre un botón que si el coche no tiene oferta ponga "*Poner en OFERTA*" y si el coche tiene oferta ponga "*Cambiar OFERTA*".



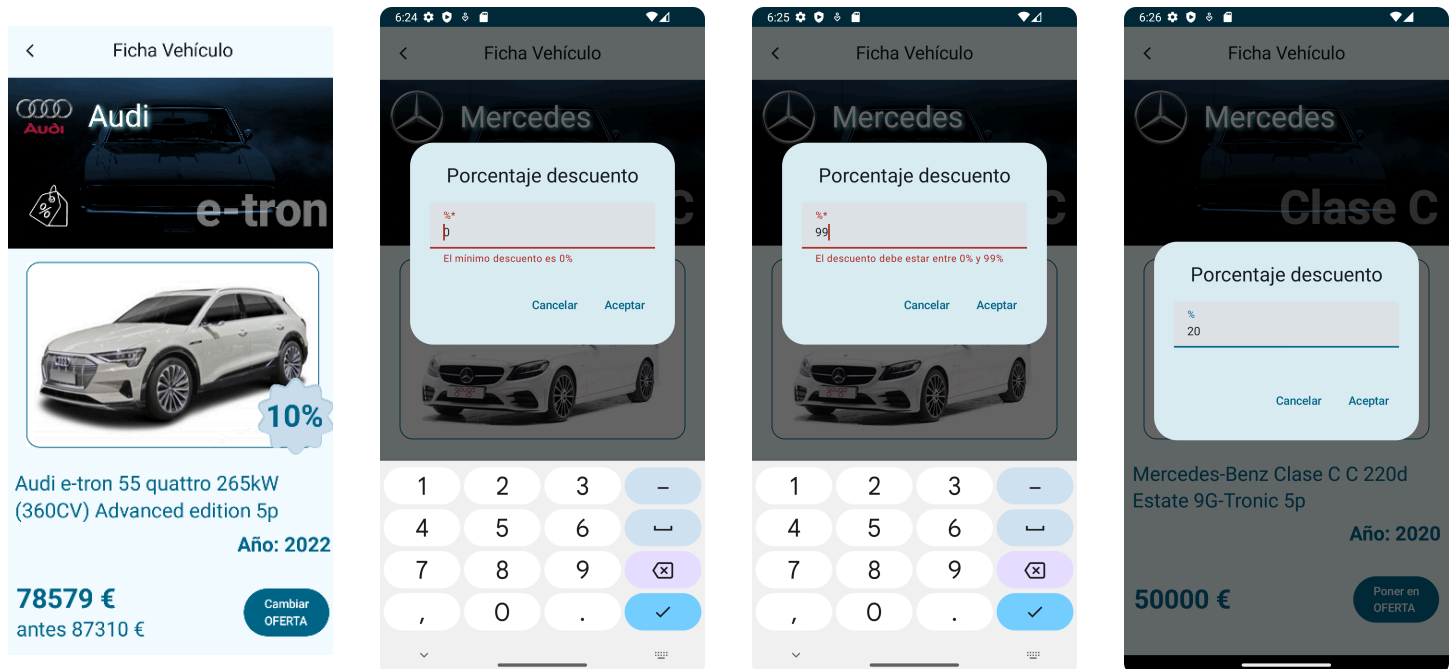
Tip

Busca los "*TODO:*" en el código de `FichaCocheScreen.kt` para saber lo que te falta por completar.

Dicho botón, llamará a un callback gestionado por el ViewModel que se encargará de navegar a un diálogo para introducir el nuevo porcentaje de descuento. De momento solo podemos llamar a este callback cuya funcionalidad ya implementaremos cuando definamos más adelante la navegación.

Tanto la pantalla de la ficha del coche como el diálogo compartirán ViewModel, que denominaremos `FichaCocheViewModel` y los eventos de ambos están ya definido en el siguiente sealed interface:

```
sealed interface FichaCochesEvent {  
    object OnCambiarOferta : FichaCochesEvent  
    object OnVolverAGaleria : FichaCochesEvent  
    data class OnAceptarDialogoDescuento(val porcentajeDescuento: Int) : FichaCochesEvent  
    object OnCancelarDialogoDescuento: FichaCochesEvent  
}
```



Definiendo la pantalla del diálogo

Para definir la pantalla del diálogo, crearemos el paquete `features/dialogodescuento` y dentro definiremos el composable `DialogoDescuento.kt`. Donde se te proporciona el siguiente composable a completar y su preview...

```
fun DescuentoDialog(
    porcentajeDescuento: Int,
    onAceptarCambios: (Int) -> Unit,
    onCancelarCambios: () -> Unit
){ // TODO }
```

Este componente debe emitir un `AlertDialog` con un `TextField` para introducir el nuevo porcentaje de descuento.

Puedes utilizar un `TextField` pero se recomienda usar `TextFieldWithErrorMessage` definido en `com.github.pmdmiesbalmis.components.ui.composables`. Para permitir solo introducir dígitos numéricos, debes establecer la propiedad `keyboardOptions` al valor `KeyboardOptions(keyboardType = KeyboardType.Number)`

Puesto que al diálogo se le pasa un valor entero con el porcentaje de descuento y su valor se modificará a través del callback `onAceptarCambios: (Int) -> Unit`. Por tanto, el componente `DescuentoDialog` será *'stateful'* y deberemos crear un estado de tipo `String` para el `TextField`, que se inicialice con el valor del porcentaje de descuento pasado como parámetro. Además, cada vez que se modifique el valor del `TextField`, se deberá actualizar el estado creado anteriormente. Para esto, se

recomienda crear una función `onPorcentajeDescuentoChange` que reciba el nuevo valor del TextField y actualice el estado creado anteriormente.

Para validar el valor introducido en el TextField, se recomienda crear un validador compuesto utilizando la clase `ValidadorCompuesto` vista en el curso y definida en

`com.github.pmdmiesbalmis.components.utils.validadores`. Este validador compuesto deberá contener un validador de texto no vacío y un validador de número entero con un rango entre 0 y 99. El resultado de la validación se deberá almacenar en un estado de tipo `Validacion` que se actualizará cada vez que se modifique el valor del TextField.

Una forma de crear el estado de validación es utilizando la función `derivedStateOf` para que se actualice automáticamente cada vez que se modifique el valor del TextField. Para esto, se recomienda crear un validador compuesto como el siguiente:

```
val validacionPorcentaje : Validacion by remember {  
    derivedStateOf {  
        validadorPorcentaje.valida(textoPorcentaje)  
    }  
}
```